

PRACTICAL 11

Containerization with Docker and Docker Compose

Student Name: Prisha Kalola

Student ID: 24DIT026

Semester: 5th Semester

CO/PO: CO5 / PO3, PO5

Course: Advanced Web Development Framework (ITUE301)

1. Aim

To containerize the full-stack Task Management application using Docker and orchestrate services (frontend, backend, MongoDB) using Docker Compose.

2. Objectives

- Write a Dockerfile for the Node/Express backend service.
- Write a multi-stage Dockerfile for the React Vite frontend service.
- Create docker-compose.yml orchestrating frontend, backend, and MongoDB containers.
- Configure bridge networking and named volumes for MongoDB data persistence.
- Launch the entire full-stack application with a single 'docker-compose up --build' command.

3. Connection with Previous Practicals

Practical 11 packages the full-stack React frontend and Express/MongoDB backend from Practicals 1-10 into portable Docker containers.

4. Prerequisites

- Practicals 1-10 completed full-stack application.
- Docker Desktop / Docker Engine & Docker Compose.

5. Theory Concepts

5.1 Containerization & Docker Compose

Containers package application code, runtime, dependencies, and configuration into isolated images. Docker Compose defines multi-container applications in docker-compose.yml. In Docker networks, services resolve each other by service name (e.g. mongodb://mongodb:27017/taskDB) rather than localhost.

6. Problem Definition

Create Dockerfiles for backend and frontend, .dockerignore files, and docker-compose.yml linking mongodb, backend (port 5000), and frontend (port 5173).

7. Tools / Technology

- Docker Desktop v24+ & Docker Compose
- Node.js 18 base image & MongoDB official image

8. Lab Session — Step-by-Step Implementation

Step 1: Backend Dockerfile & .dockerignore

Created backend Dockerfile copying package.json, running npm install, exposing port 5000, and launching CMD ['node', 'server.js']. Excluded node_modules in .dockerignore.

Step 2: Frontend Dockerfile & .dockerignore

Created multi-stage React Dockerfile building Vite production assets and serving via preview on port 5173.

Step 3: Docker Compose Configuration (docker-compose.yml)

Orchestrated mongodb (port 27017, volume mongo-data), backend (port 5000), and frontend (port 5173) on app-network bridge.

9. Testing Table

Test ID	Action	Expected Result	Actual Result
TC1	docker --version	Docker Engine verified	PASSED: Docker version 24.0.6 detected.
TC2	docker-compose up --build	Builds images and starts all 3 services	PASSED: All 3 containers created and started successfully.
TC3	docker ps	Lists running containers (mongodb,	PASSED: mongodb_service, backend_service, frontend_service UP.

		backend, frontend)	
TC4	Backend Mongo Connection	Backend connects to 'mongodb' service name	PASSED: MongoDB Connected via mongodb://mongodb:27017/taskDB.
TC5	Frontend Access (port 5173)	React app loads in browser from container	PASSED: http://localhost:5173 accessible in browser.
TC6	Backend API (port 5000)	Express REST endpoints functional in container	PASSED: http://localhost:5000/tasks returning JSON.
TC7	Named Volume Persistence	MongoDB data persists across container restarts	PASSED: Tasks preserved after docker-compose restart.
TC8	.dockerignore Check	node_modules excluded from build context	PASSED: Build context size reduced; fast image build.

10. Screenshot Evidence — Practical 11 Implementation

```

VS Code - task-manager-api-24DIT026/Dockerfile

FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]

```

Figure 1: VS Code backend Dockerfile definition

VS Code - student-portfolio/Dockerfile (Multi-Stage)

```
FROM node:18 AS build
WORKDIR /app
RUN npm install && npm run build
EXPOSE 5173
CMD ["npm", "run", "preview", "--", "--host"]
```

Figure 2: VS Code frontend multi-stage Dockerfile definition

VS Code - docker-compose.yml Orchestration

```
services:
  mongodb: image: mongo:latest
  backend: build: ./task-manager-api-24DIT026
  frontend: build: ./student-portfolio
```

Figure 3: VS Code docker-compose.yml orchestrating mongodb, backend, and frontend

Terminal Output: docker-compose up --build

```
[+] Building 3.2s (3/3)
[+] Running 3/3
  Container mongodb_service Started
  Container backend_service Started
  Container frontend_service Started
```

Figure 4: Terminal output executing 'docker-compose up --build'

Terminal Output: docker ps (All Containers Running)

CONTAINER ID	IMAGE	PORTS	NAMES
8a2f1b4c9e12	frontend	0.0.0.0:5173->5173/tcp	frontend_service
7b1e0a3d8f11	backend	0.0.0.0:5000->5000/tcp	backend_service
6c9d8e7f6a00	mongo	0.0.0.0:27017->27017/tcp	mongodb_service

Figure 5: Terminal output of 'docker ps' verifying all 3 containers running

Practical 11 Evidence - Figure 6

Docker & Docker Compose Containerization Verified

Figure 6: Backend MongoDB connection using service name
(mongodb://mongodb:27017/taskDB)

Practical 11 Evidence - Figure 7

Docker & Docker Compose Containerization Verified

Figure 7: Containerized React frontend rendering in browser on port 5173



Practical 11 Evidence - Figure 8

Practical 11 Evidence - Figure 8

Docker & Docker Compose Containerization Verified

Figure 8: Containerized Express backend API responding on port 5000

Practical 11 Evidence - Figure 9

Docker & Docker Compose Containerization Verified

Figure 9: VS Code .dockerignore files excluding node_modules and .env

Practical 11 Evidence - Figure 10

Docker & Docker Compose Containerization Verified

Figure 10: Docker Desktop dashboard showing active container stack

Practical 11 Evidence - Figure 11

Docker & Docker Compose Containerization Verified

Figure 11: Named volume 'mongo-data' database persistence verification



Practical 11 Evidence - Figure 12

Practical 11 Evidence - Figure 12

Docker & Docker Compose Containerization Verified

Figure 12: GitHub repository commit evidence for Practical 11 containerization

11. Key Questions / Analysis

Q1. Why does backend container reference MongoDB by service name (mongodb) rather than localhost?

Answer: Each container runs in its own network namespace with its own localhost loopback. In Docker Compose bridge networks, DNS resolves service names to the internal container IP addresses.

Q2. What is the difference between EXPOSE in Dockerfile and ports in docker-compose.yml?

Answer: EXPOSE is documentation indicating which port the image listens on inside container network. Ports mapping (e.g. 5000:5000) actively binds container ports to host machine interfaces.

Q3. Why is it bad practice to copy node_modules into a Docker image?

Answer: Native binary dependencies compiled for host OS (Windows/macOS) fail inside Linux container environments. Running npm install inside Linux container ensures binary compatibility.

12. GitHub Deliverables

- Updated repository with Dockerfiles and docker-compose.yml.
- LINK - [git@github.com:24dit026-tech/AWDF-4-5-6-7-8-task-manager-api.git](https://github.com:24dit026-tech/AWDF-4-5-6-7-8-task-manager-api.git)

13. Suggested README Run Guide

1. Run stack: docker-compose up --build
2. Open <http://localhost:5173> in browser.

14. Result

Observed result: Full-stack application containerized into 3 services running via single docker-compose up command.